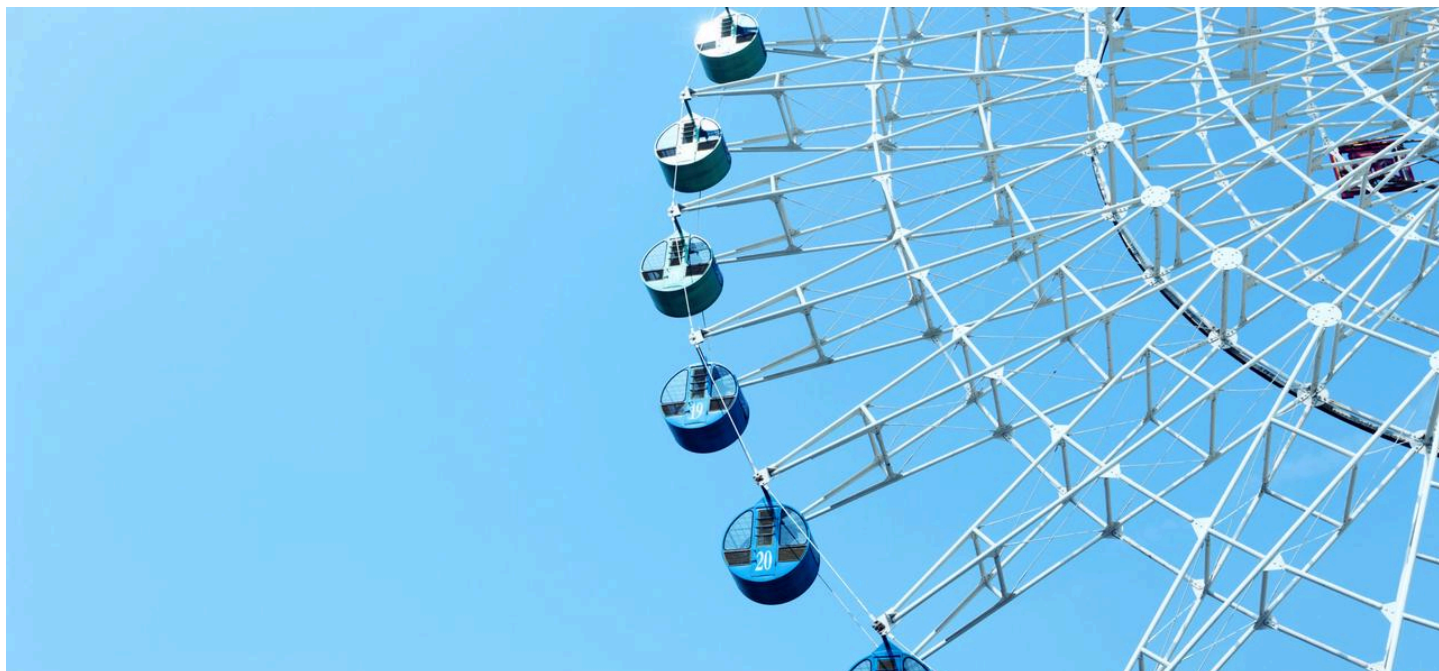




0-intro

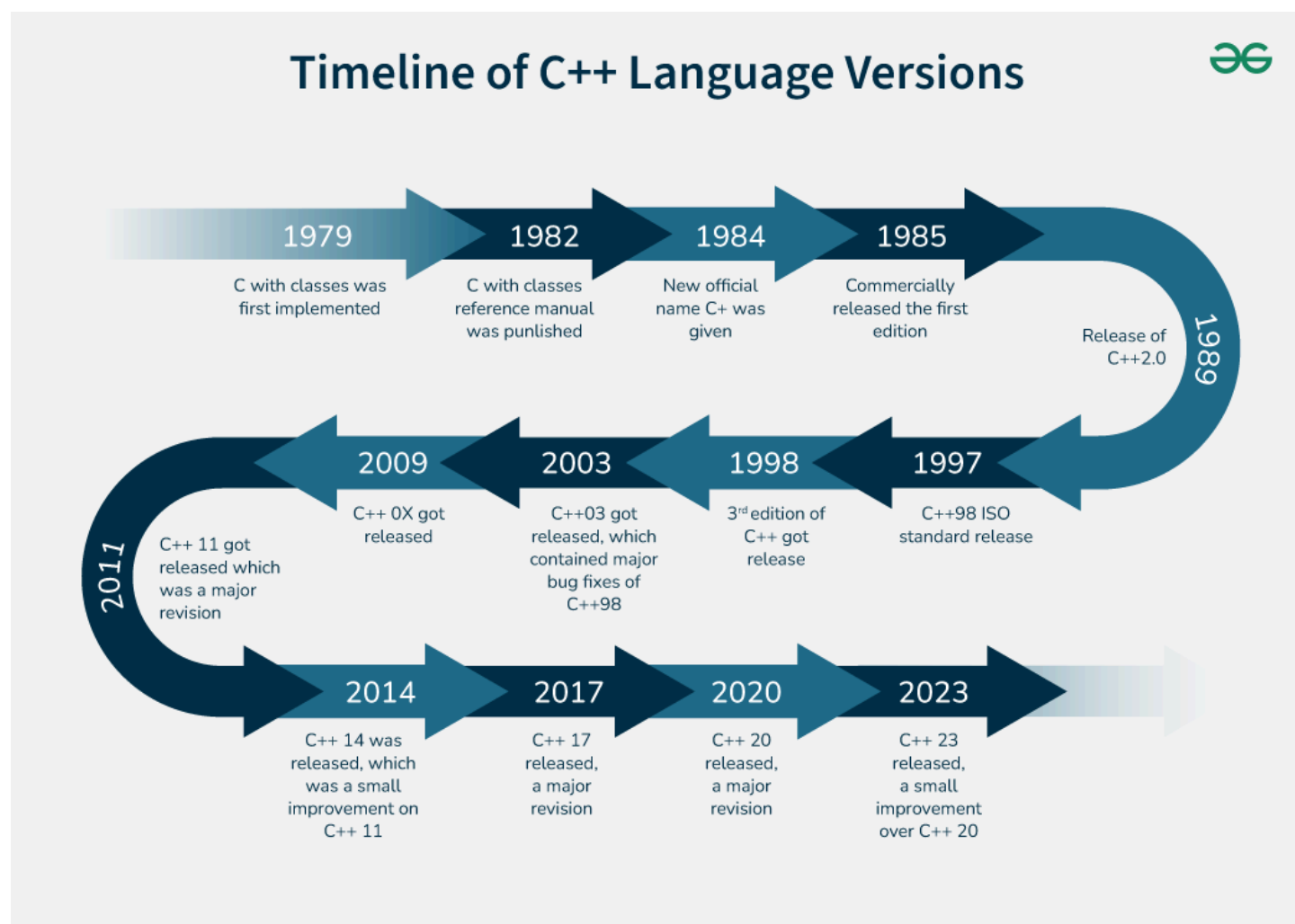
1. 简介

本节课简要介绍 CPP 课程与 C++ 语言，帮助大家了解课程概况、C++ 语言标准、学习资源与开发工具，并动手完成第一个程序。

- **主旨：**了解课程基本情况，认识 C++ 语言标准，熟悉开发工具，并动手编写、编译、运行第一个 C++ 程序。
- **学习目标：**
 - 了解 CPP 课程的基本情况（课程定位、教学内容等）；
 - 了解 C++ 语言的发展简史与标准演进（C++98、C++11、C++14、C++17、C++20、C++23 等）；
 - 了解教材与参考书籍；
 - 掌握使用 VS Code 创建 C++ 项目，编译并运行第一个程序（Hello World）；
 - 理解计算思维三大结构（顺序、选择、循环），并实现猜数游戏；
- **重点：**使用 VS Code 创建 C++ 项目并运行 Hello World 程序；计算思维的三大结构及其 C++ 语言实现。
- **难点：**将自然语言描述的问题逐步转化为程序结构。

2. C++ 语言标准

History of C++ @ [cppreference](#) 总结了 C++ 语言的发展简史。其中，C++98 (ISO/IEC 14882:1998) 是第一个正式发布为国际标准的 C++ 版本；此后经历了 C++03、C++11、C++14、C++17、C++20、C++23 等版本。截止到 2026 年 9 月，最新已正式发布的 C++ 标准是 C++23，即 ISO/IEC 14882:2024 (2024 年 10 月由 ISO 正式发布)。C++26 的技术工作已于 2026 年 3 月完成，目前处于 ISO 的最终批准阶段 (ISO/IEC DIS 14882)，预计将以 ISO/IEC 14882:2026 正式发布。



C++11 与 C++20 是两个里程碑式的重大修订：C++11 引入了移动语义、lambda 表达式、智能指针、并发支持等现代 C++ 的核心特性，C++20 则引入了概念 (concepts)、范围 (ranges)、协程 (coroutines)、模块 (modules) 等大型特性。C++11 与之前的 C++98/03 有很大差别，通常被认为是现代 C++ (Modern C++) 的开端。

i 本学期，我们主要采用 C++20 标准，并适时介绍一些 C++23、C++26 中的特性。

💡 如需修改 C++ 标准：本课程统一使用 C++20。如需改用其他标准（如 C++23），把编译命令中的 `-std=c++20` 改为对应版本即可；CMake 项目中需同步设置 `CMAKE_CXX_STANDARD`。GCC 16 及以上不指定标准时默认 C++20。

作为一名合格的 C++ 程序员，你需要了解 C++ 的演变逻辑以及一些关键特性在各个版本中的演变过程。但是，你不需要成为一个语言律师（[Language Lawyer](#)；当然，这其中也充满乐趣），不要死记硬背很多语言的细节。对于那些你需要了解的演变过程，我们会在之后的课堂中着重强调。

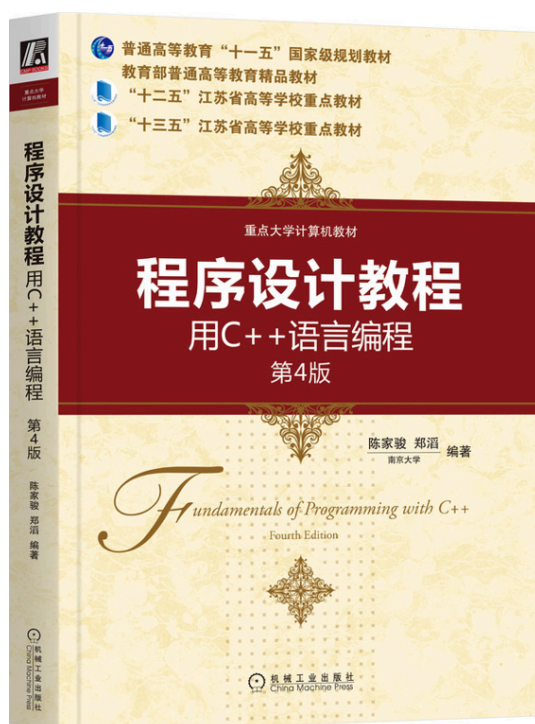
! C++ 最初是从 C 语言发展而来的，但与 C 已有巨大差别，你不应该用 C 语言的经验编写 C++ 代码。

若想了解更多关于 C++ 语言的发展历史，可以参考“C++ 语言之父”[Bjarne Stroustrup](#)发表的论文《[A History of C++: 1979–1991](#)》、书籍《[The Design and Evolution of C++](#)》与《[A Tour of C++](#)》，以及 ISO C++ 标准委员会 [WG21](#) 的官方页面与 [The Standard C++ Foundation](#) ([isocpp.org](#)) 网站。

3. 教材与参考书籍

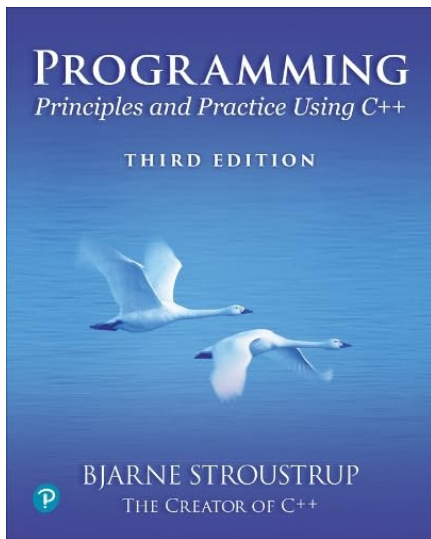
3.1 教材

本课程指定教材如下，请认准“第 4 版”。

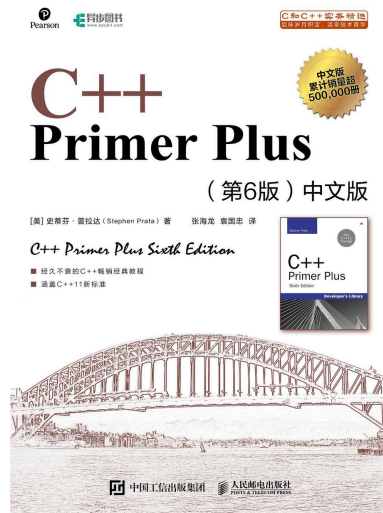


3.2 参考书籍

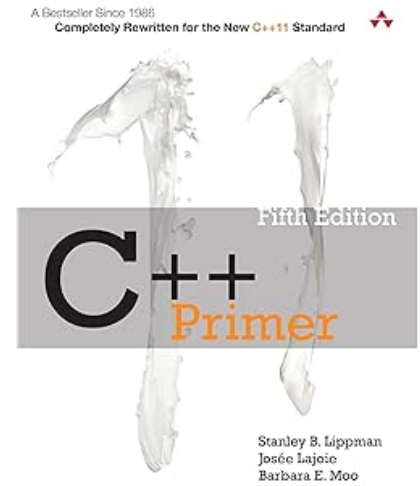
3.2.1 入门书籍



Programming (PPP3):
Programming: Principles
and Practice using C++ (3rd
Edition); 基于 C++20 标准

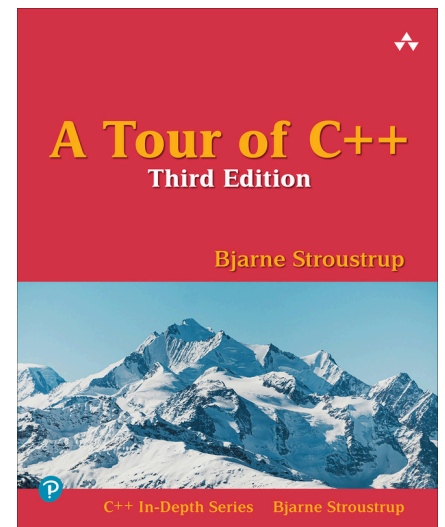


C++ Primer Plus (很遗
憾，2020 年出版的第 6 版
仍是基于 C++ 11 标准)



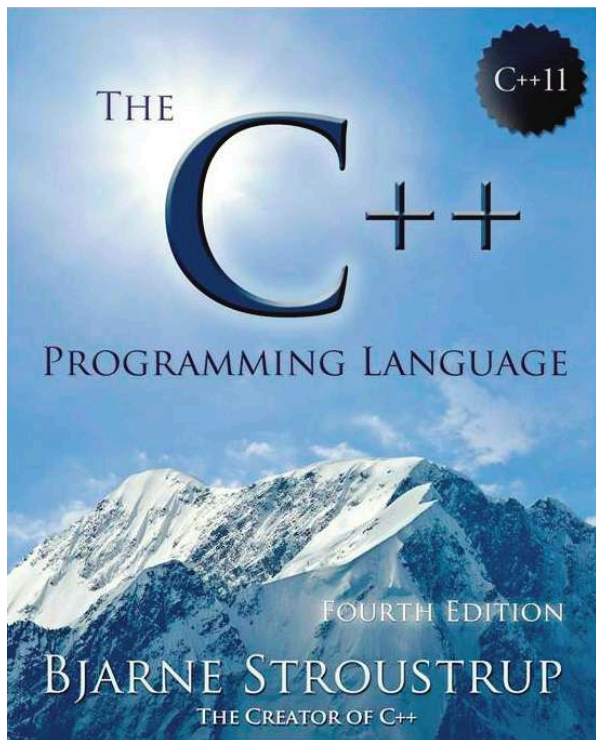
C++ Primer (很遗憾，这是第五版，
基于 C++ 11 标准)。
第六版已于 2026 年 3 月份出版，但
难以购买。

3.2.2 中级书籍

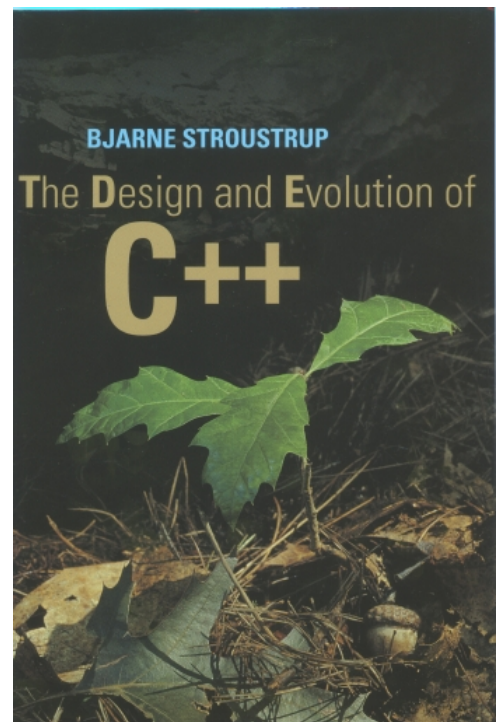


Tour++: A Tour of C++

3.2.3 高阶书籍



TC++PL: The C++ Programming Language



D&E: The Design and Evolution of C++

3.3 不推荐书单

！ 有些教材“有毒”，不要读，尤其是那些为了评奖、评职称、项目交差编撰出来的教材，也包括某些被纳入国家规划的教材。它们可能东拼西凑、组织混乱、内容陈旧、避重就轻、误人子弟、贻害无穷。如果你判断不了，可以与我联系。

这里就不点名了。

4. Hello World

📖 本课程使用 [VS Code](#) 作为开发环境：请先参考 [📖 Compilers](#) 讲义安装、配置编译器，参考 [📖 VS Code](#) 讲义安装、配置 VS Code，并创建、编译、运行 C++ 项目。

在 VS Code 中新建源文件 `hello.cpp`，输入如下代码：

代码块

```
1 // File: hello.cpp
2 // Author: hfwei
3 // Date: Sep. 12, 2026
4
5 #include <iostream>
6
7 /*
```

```

8  * "main" is a function which accepts no arguments (thus "()" here)
9  * and returns an integer (thus "int" here).
10 * In C++, the idiomatic form is "int main()" (no "(void)").
11 */
12 int main() { // single-line comments can be put here
13     std::cout << "Hello World\n";
14
15     return 0;
16 }

```

4.1 注释

C++ 语言支持两种注释：以 `//` 开头的单行注释（single-line comment）以及 `/*` 与 `*/` 之间的多行注释（multi-line comment）。

注释是写给人看的，编译器在编译代码时会忽略所有的注释。

好的代码是自解释的/自明的（self-explanatory），无需过多注释。

好的注释与好的代码应是相辅相成的关系。

4.2 main 函数

在本学期的绝大部分时间里，`main` 函数的形式都如下所示：

代码块

```

1  int main() {
2      // Your Code Here ...
3
4      return 0;
5  }

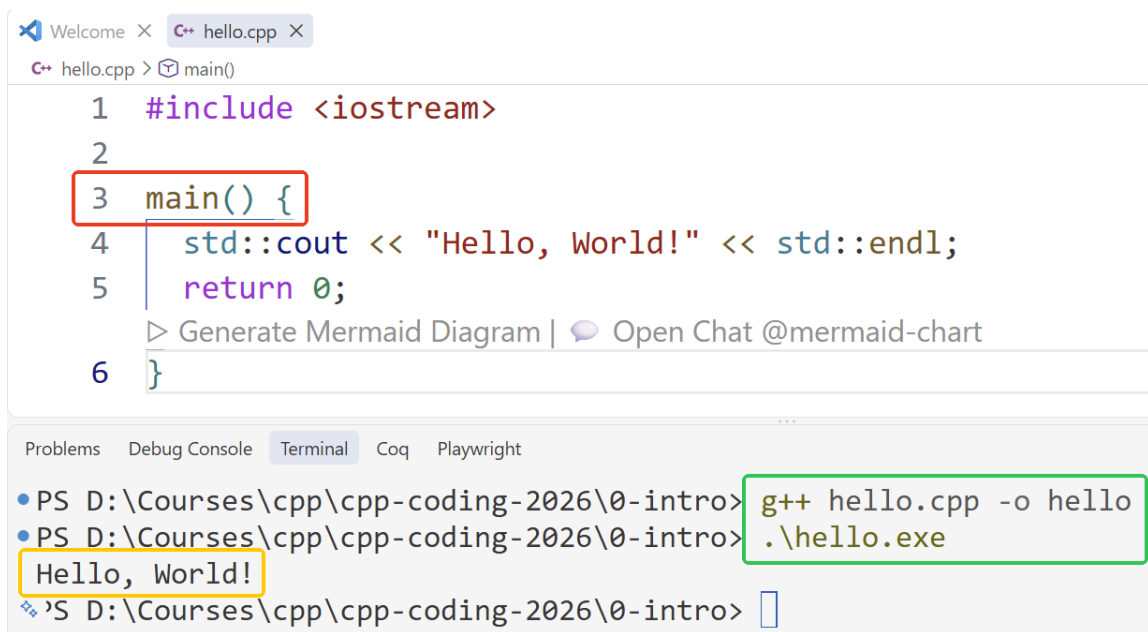
```

- `main` 是函数名。`main` 函数是 C++ 程序的入口（entry）函数。



初学者常犯的第一个错误就是将 `main` 拼写成 `mian` 或 `man`。

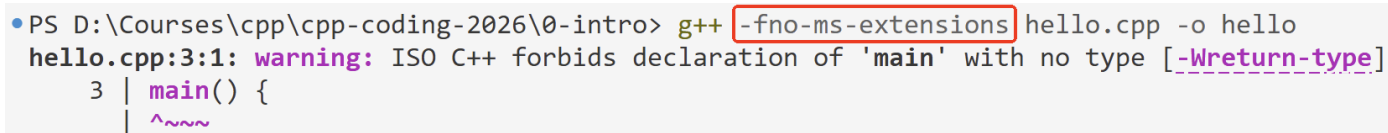
- `mian` 函数不接受任何参数，其后的空括号 `()` 即表示“无参数”。
- `{` 与 `}` 之间的代码称为“函数体”（function body）。
- `int main` 中的 `int` 表示该函数的返回值类型必须是“整型（integer）”。C++ 标准规定，函数返回值类型 `int` 不可省略。为了验证这一点，让我们尝试删除 `int`，然后再次编译：



```
1 #include <iostream>
2
3 main() {
4     std::cout << "Hello, World!" << std::endl;
5     return 0;
6 }
```

```
PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ hello.cpp -o hello
PS D:\Courses\cpp\cpp-coding-2026\0-intro> .\hello.exe
Hello, World!
```

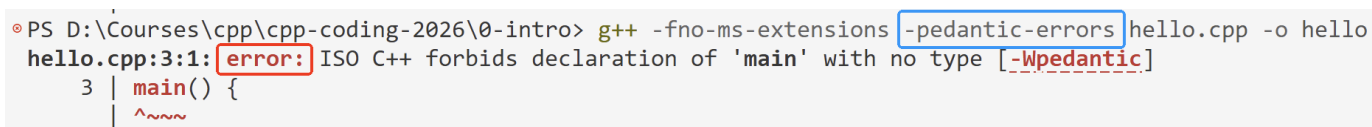
等等……怎么仍然能够正常编译、运行？`g++` 连如此简单的程序都处理不了吗？当然不是，这是历史原因造成的。C++ 是从 C 语言发展而来的，早期的 C 语言允许隐式的 `int` 函数返回值类型。C++ 为了保持向后兼容（让人又爱又恨的向后兼容），很多编译器都在默认情况下允许这种隐式的 `int`（implicit int）。不仅如此，Microsoft 的 MingW 可能还添加了更多扩展……



```
PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ -fno-ms-extensions hello.cpp -o hello
hello.cpp:3:1: warning: ISO C++ forbids declaration of 'main' with no type [-Wreturn-type]
 3 | main() {
   | ~~~~
```

如上所示，添加 `-fno-ms-extensions` 编译选项，禁用 Microsoft 扩展，可以让编译出现警告信息 `warning: ISO C++ forbids declaration of 'main' with no type`。

然而，warning 并不阻止编译器继续编译。可以使用 `-pedantic-errors` 将 warning 提升为 error:



```
PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ -fno-ms-extensions -pedantic-errors hello.cpp -o hello
hello.cpp:3:1: error: ISO C++ forbids declaration of 'main' with no type [-Wpedantic]
 3 | main() {
   | ~~~~
```

 本课程始终提倡编写符合 C++ 语言标准的代码。为此，建议始终为 `g++` 命令添加如下编译选项：

`-Wall -Wextra -pedantic-errors -fno-ms-extensions`。其中，

- `-Wall`：开启一组“常见且有用的警告”
- `-Wextra`：开启一些没有包含在 `-Wall` 中的额外警告
- `-pedantic-errors`：将警告提升为错误
- `fno-ms-extensions`：禁用 MS 扩展

- `return 0` 是返回语句，它返回 `0`，符合 `int main` 中的类型规定。

- C++ 标准规定，如果 `main` 函数是正常结束的，则返回 `0`，否则返回非零值。
- C++ 标准规定，若 `main` 函数执行到末尾而没有显式返回语句，则等价于 `return 0`。因此，`return 0` 可以省略，但仍建议遵循 C++ 标准中的样例，保留 `return 0`。
- `main` 函数由操作系统负责调用，函数执行结束，返回值也是返回给操作系统。操作系统可以根据返回值判断 `main` 函数是否正常结束，并决定下一步做什么。

4.3 Hello World\n 字符串

`main` 函数体内 `std::cout << "Hello World\n";` 中的 `Hello World\n` 是字符串 (string)。(更严格地讲，它是字符串字面量 (string literal)，与用变量指代的字符串相对)。字符串是 C++ 中的一种常用且重要的类型，后续会进行专题讲解。

这里只需要注意两点：第一，字符串的两端要加双引号；第二，`\n` 是换行符 (newline character)，是转义序列 (escape sequence) 的一种。所谓转义，就是指这里的 `n` 不再代表字母 `n`，而是 `\n` 作为一个整体代表换行符。例如，`"Hello World\n"` 表示：`Hello World` 本身，后跟一个“将光标移到下一行行首”的语义。

? 如果字符串本身含有双引号，怎么办？如果字符串本身含有 `\n` 这两个字符，怎么办？

4.4 cout 输出

`cout` 是 C++ 标准库提供的标准输出流 (standard output stream) 对象。目前，你不需要理解什么是对象，你只需要将 `cout` 理解成计算机的屏幕。`cout` 是 *character output* (字符输出) 的缩写。简单地说，`cout` 负责把程序中的信息输出 (显示) 到屏幕上。

使用 `cout` 输出内容时，需要配合插入运算符 (insertion operator) `<<`。`<<` 的作用是把右侧的内容“插入”到输出流中，从而显示在屏幕上。例如：

代码块

```
1 std::cout << "Hello World\n";
```

会输出字符串 `Hello World` 并换行。可以连续使用多个 `<<`，把多项内容依次输出，例如：

代码块

```
1 std::cout << "The answer is " << 42 << ".\n";
```

输出结果为：`The answer is 42.` (换行)。

除了 `\n`，C++ 还提供了另一种换行方法：`std::endl`。`endl` 是 *end line* (结束一行) 的缩写，是一个操纵符 (manipulator)。它的用法与 `\n` 类似：

代码块

```
1 std::cout << "Hello World" << std::endl;
```



`std::endl` 与 `\n` 的区别在于, `std::endl` 在换行的同时, 还会刷新输出缓冲区 (flush the output buffer), 即强制把尚未显示的内容立即输出到屏幕; 而 `\n` 只负责换行, 不会刷新缓冲区。

在大多数简单程序中, 两者的输出效果相同。 `\n` 写起来更简洁、效率也更高, 因此更常用; 在需要立即看到输出 (如调试时) 或交互式程序中, 可以使用 `std::endl`。

4.5 std 命名空间

`std::cout`、`std::endl` 中的 `std::` 是什么意思? 这涉及到命名空间 (namespace) 的概念。

程序开发经常需要多人合作, 在给程序中的实体命名时, 难免会起冲突。命名空间是 C++ 用来组织名字、避免名字冲突 (name collision) 的机制。可以将命名空间理解成层次式的文件系统, 虽然同一目录下的同名文件会起冲突, 不同目录下的同名文件可以相安无事。

`std` (standard) 是 C++ 标准库使用的命名空间。在 `std::cout` 中, `::` 是作用域解析运算符 (scope resolution operator)。`std::cout` 表示 "命名空间 `std` 中的 `cout`", 它告诉编译器到命名空间 `std` (类比于目录 `std`) 中去找 `cout` (类比于文件 `cout`)。

如果不用 `std::`, 会发生什么呢? 我们考查两种情况。

第一种, 将 `std::cout` 写成 `::cout`:

```
1 #include <iostream>
2
3 int main() {
4     ::cout << "Hello, World!\n";
5     return 0;
6 }
```

the global scope has no "cout"

Problems | Debug Console | Terminal | Coq | Playwright

PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ -fno-ms-extensions -pedantic-errors hello.cpp -o hello

hello.cpp: In function 'int main()':

hello.cpp:4:5: error: '::cout' has not been declared; did you mean 'std::cout'?

```
4 |     ::cout << "Hello, World!\n";
  |     ^~~~~
  |     std::cout
```

In file included from hello.cpp:1:

C:/msys64/ucrt64/include/c++/16.2.0/iostream:65:18: note: 'std::cout' declared here

```
65 | extern ostream cout;          ///< Linked to standard output
    |                  ^~~~~
```

黄色框（MinGW-w64 `g++`）中的错误信息表明，编译器找不到它想要的 `std::cout`，它认为 `::cout` 未被声明（declared）过。图中红色框（MSVC 的 IntelliSense）给出了更具体的错误信息：`the global scope has no "cout"`，指明编译器在全局（`global`）命名空间中没有找到 `cout`。这是因为，单独的 `::` 表示全局命名空间。因此，编译器尝试在全局命名空间中查找 `cout`，但没有找到。

第二种，将 `std::cout` 写成 `cout`：



```
1 #include <iostream>
2
3 int main() {
4     cout << "Hello, World!\\n";
5     return 0;
6 }
```

identifier "cout" is undefined

Problems | Debug Console | Terminal | Coq | Playwright

PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ -fno-ms-extensions -pedantic-errors hello.cpp -o hello

hello.cpp: In function 'int main()':

hello.cpp:4:3: error: 'cout' was not declared in this scope; did you mean 'std::cout'?

4 | cout << "Hello, World!\\n";

std::cout

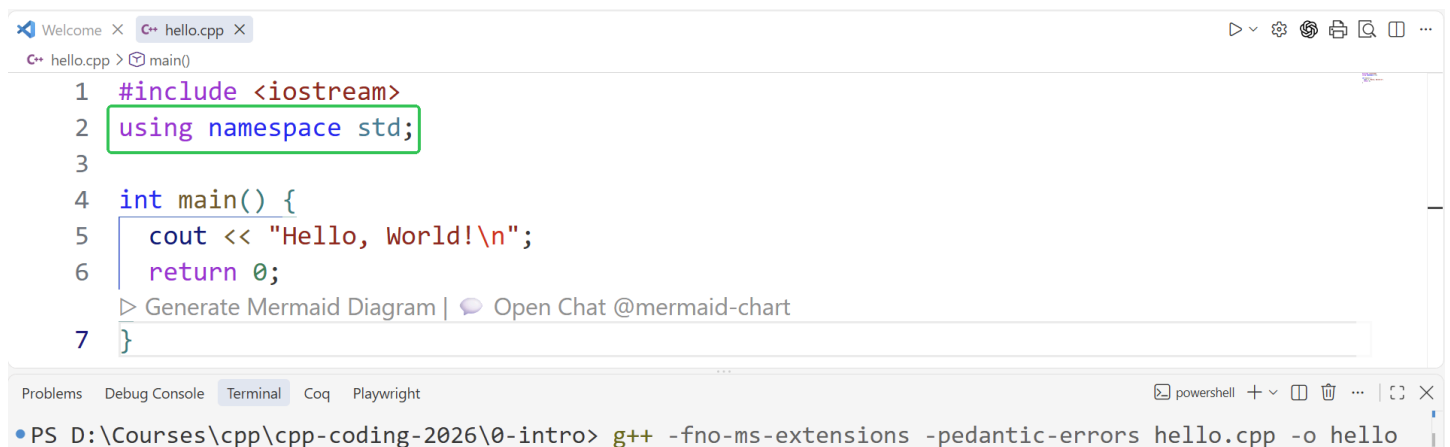
In file included from hello.cpp:1:

C:/msys64/ucrt64/include/c++/16.2.0/iostream:65:18: note: 'std::cout' declared here

65 | extern ostream cout; // Linked to standard output

同样的错误，然而编译器查找 `cout` 的具体过程有所不同。`::cout` 是仅在全局命名空间中查找，找不到则报错；`cout` 则是先在当前命名空间中查找，找不到则到向外一层命名空间中查找，依次类推，找到即停。如果到了全局命名空间还是没有找到，则报错。

有些程序员觉得每次都写 `std::` 过于繁琐，转而使用"大杀器" `using namespace std;`：



```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "Hello, World!\\n";
6     return 0;
7 }
```

Problems | Debug Console | Terminal | Coq | Playwright

PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ -fno-ms-extensions -pedantic-errors hello.cpp -o hello

然而，本课程不推荐使用 `using namespace std`，因为它将 `std` 空间中的所有名字都对（当前查找规则）开放了，容易导致名字冲突或意想不到的名字隐藏错误。

 遵循 C++ Core Guidelines SF.7 的精神，避免全局作用域使用 `using namespace`。关于命名空间的更多内容，参见 [namespaces @ cppreference](#)。

另一种折中的、尚可接受的方案是使用 `using std::cout`，仅暴露 `cout`：



The screenshot shows a C++ IDE with a file named 'hello.cpp'. The code is as follows:

```
1 #include <iostream>
2 using std::cout;
3
4 int main() {
5     cout << "Hello, World!\n";
6     return 0;
7 }
```

Below the code editor, there is a terminal window with the following command:

```
PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ -fno-ms-extensions -pedantic-errors hello.cpp -o hello
```

4.6 #include <iostream> 头文件

那么, `#include <iostream>` 是什么意思呢?

首先，`iostream` 指的是一个具体文件，称为头文件（header file），它里面包含对 `cout` 对象的声明。其次，`#include` 是预处理指令（preprocessor directive），所谓预处理，就是在编译之前所做的操作。顾名思义，`#include <iostream>` 就是将 `iostream` 中的内容复制到当前文件 `hello.cpp` 当前行所在的位置，因此对 `cout` 的声明也被包含到 `hello.cpp` 文件中了，接下来才能正常使用 `std::cout`。

在 `hello.cpp` 中，如果将 `#include <iostream>` 注释掉，则会出现如下错误：

```
1 // #include <iostream>
2
3 int main() {
4     std::cout << "Hello, World!\\n";
5     return 0;
6 }
```

namespace "std" has no member "cout"

Generate Mermaid Diagram | Open Chat @mermaid-chart

Problems Debug Console Terminal Coq Playwright

PS D:\Courses\cpp\cpp-coding-2026\0-intro> g++ -fno-ms-extensions -pedantic-errors hello.cpp -o hello

hello.cpp: In function 'int main()':

hello.cpp:4:8: error: 'cout' is not a member of 'std'

```
4 |     std::cout << "Hello, World!\\n";
  |           ^~~~~
hello.cpp:1:1: note: 'std::cout' is defined in header '<iostream>'; this is probably fixable by adding '#include <iostream>'
+++ |+#include <iostream>
1 | // #include <iostream>
```

4.7 `iostream` 与标准库

4.7.1 `iostream`

`iostream` 是 `input/output stream` 的缩写。其中，`input` 意为“输入”，`output` 意为“输出”。输入、输出是计算机程序与外界进行交互的方式。程序通过输入机制从外界获取数据，对

数据进行处理，然后将处理结果通过输出机制展示给外界。C++ 标准库将输入输出相关的功能集中放在 `<iostream>` 头文件中，例如 `std::cout` 与 `std::cin` 分别是标准输出流、标准输入流对象。

 [1-types-io](#) 将进一步介绍 `std::cout` 与 `std::cin`。

4.7.2 标准库

标准库（standard library）是 C++ 标准（ISO/IEC 14882）规定的一组现成功能：常用的类型、函数、对象等，以头文件的形式提供给程序员直接使用。C++ 语言本身与标准库共同构成了完整的 C++：语言规定了“怎么写”，标准库提供了“现成能用”的零件。

标准库涉及的范围很广，例如：输入输出（`<iostream>`）、字符串（`<string>`）、常用容器（如 `<vector>`、`<map>`）、算法（如 `<algorithm>` 中的 `std::sort`）、随机数（`<random>`）、时间（`<chrono>`）、文件操作、多线程与智能指针等。

标准库非常重要：它是经过长期打磨的、正确且高效的代码，使用标准库可以避免“重复造轮子”，让程序更可靠、更易读、更易移植。

 学习 C++ 的一个重要方面，就是学会使用标准库。本课程会带大家尽早了解并使用标准库。

5. Guessing the Number

*Language
is the dress
of thought.*

SAMUEL JOHNSON

语言是一种工具。我们使用语言表达我们的思维。从这个角度来说，C++ 语言是“术”，解决问题的思维才是根本之“道”，正所谓“道为术之灵，术为道之体；以道统术，以术得道”。对于任何问题，动手写代码之前，你先要能用自然语言描述你解决该问题的方案。

下面，我们以[猜数游戏](#)为例，介绍解决问题的计算思维。

5.1 多文件项目结构

首先，在 `cpp-coding-2026\0-intro` 文件夹下创建一个新的源文件，名为 `guess.cpp`，通过以下命令进行编译、运行：

代码块

```
1  g++ -o guess guess.cpp
2  .\guess.exe
```

5.2 计算思维

所谓“计算思维”，就是像计算机一样思考、解决问题。计算机是如何思考问题的呢？计算机遵循三种朴素的思维结构：顺序结构、选择结构与循环结构。像计算机一样思考问题就是要使用这三种思维结构来构建你的问题解决方案：首先，你要能快速识别给定问题中的这三种结构；其次，你要能将这三种结构有机地组合在一起；最后，你要能熟练地使用 C++ 语言实现它们。

如下代码段用注释的形式展示了实现猜数游戏的 6 个步骤。

代码块

```
1  // File: guess.cpp
2
3  int main() {
4      /*
5       * (1) Display the game rule
6       */
7
8      /*
9       * (2) generate a secret number
10     */
11
12     /*
13      * (3) ask the player to enter a number
14      */
15
16
17     /*
18      * (4) receive the guessed number
19      */
20
21     /*
22      * (5) compare the guessed number with the secret number
23      * and inform the player of the result
24      */
25
26     /*
27      * (6) loop: repeat until the player wins or loses
```

```
28     */
29
30     return 0;
31 }
```

5.2.1 顺序结构

猜数游戏按照先后包含如下步骤：

- (1) 向用户展示游戏规则
- (2) 计算机产生一个秘密数
- (3) 提示用户输入数
- (4) 计算机接收用户输入的数
- (5) 计算机将用户输入的数与秘密数进行比较并将比较结果告知用户
- (6) 重复 (3) 到 (5) 步直到游戏结束。

这种自然的先后关系就对应于程序设计语言中的顺序结构。

5.2.2 选择结构

当计算机接收到用户输入的猜测数 `guess` 后，需要将 `guess` 与 `secret` 进行比较，并根据比较的结果执行不同的动作。为此，C++ 语言提供了 “（条件）选择” 结构：

代码块

```
1  if (condition) { // if condition holds,
2      // then do something
3  } else {          // else
4      // do something else
5  }
```

对于猜数游戏，`guess` 与 `secret` 比较有三种结果。为了表达这种 “三路”（three-way）选择，我们需要在 `else` 后面再嵌套一个 `if ... else ...`。代码如下所示：

代码块

```
1      /*
2      * (5) compare the guessed number with the secret number
3      * and inform the player of the result
4      */
5      if (guess == secret) { // '=' means 'equals'; it is not '==' !
6          std::cout << "You Win!\n";
7      } else if (guess > secret) {
8          std::cout << "guess > secret\n";
```

```

9      } else {
10         std::cout << "guess < secret\n";
11     }

```

5.2.3 循环结构

日子周而复始，人生代代相传。不管情不情愿，你每天都在做大量重复性的工作。在程序设计中，我们使用循环（Loop）结构表达这种重复。任何一个循环结构都包含两大要素：一是循环体（loop-body），二是循环条件（loop-condition）。

首先，对于猜数游戏，循环体包含（4）、（5）两个步骤，用 C++ 语言表达如下。 `while (condition) { loop-body }` 语句表示只要 `condition` 还成立，就重复执行 `loop-body`，直到 `condition` 不再成立为止。

代码块

```

1      int main() {
2          // ...
3
4          /*
5           * loop: repeat until the player wins or loses
6           */
7          while (condition) { // while condition holds, repeatedly do the following
                                things:
8              /*
9               * (4) receive the guessed number
10              */
11
12             /*
13              * (5) compare the guessed number with the secret number
14              * and inform the player of the result
15              */
16         }
17
18         return 0;
19     }

```

接下来，要确定 `condition` 是什么，也就是游戏结束的条件。我们预设了最大可以尝试的次数 `tries`，因此一个简单的条件是，只要次数还没有问完，循环就可以继续。相应的 C++ 语言实现如下所示。

代码块

```

1      int main() {
2          // ...

```

```

3      int tries = 7;
4
5      /*
6       * loop: repeat until the player wins or loses
7       */
8      while (tries > 0) { // as long as the player still has chances, repeatedly
do the following things:
9          /*
10         * (4) receive the guessed number
11         */
12
13         /*
14         * (5) compare the guessed number with the secret number
15         * and inform the player of the result
16         */
17         if (guess == secret) { // '==' means 'equals'; it is not '=' !
18             std::cout << "You Win!\n";
19         } else if (guess > secret) {
20             std::cout << "guess > secret\n";
21         } else {
22             std::cout << "guess < secret\n";
23         }
24
25         tries--; // decrement tries by one (equivalent to tries = tries - 1;)
26     }
27
28     return 0;
29 }

```

然而，该程序有一个 bug：如果玩家运气够好，并不需要用完 `tries` 次就能猜出秘密数来。这种情况下，虽然内层的 `if` 已经判断出游戏可以结束，但是由于 `tries > 0` 仍然成立，`while` 循环还会继续，这不是我们想要的。我们期望当玩家猜中数后，游戏立即结束。也就是说，我们希望能提前终止这个循环。为此，C++ 语言提供了 `break` 语句，`break` 语句能立刻“打破”套在外面的一层循环，令代码下一步改为执行该循环后方的语句（而不是原定的下一行代码）。完整代码参考下一节（尤其注意 `break` 语句所在的行）。

5.3 完整代码

完整的猜数游戏代码如下，请体会其中所表达的顺序结构、选择结构与循环结构。

代码块

```

1  // File: guess.cpp
2
3  #include <iostream>
4  #include <random>

```

```
5
6  int main() {
7      /*
8       * (1) Display the game rule
9       */
10     constexpr int max = 100;
11     int tries = 7;
12
13     std::cout << "The computer will generate a number between 1 and " << max <<
14     ".\n";
15     std::cout << "You have " << tries << " chances.\n";
16
17     /*
18     * (2) generate a secret number
19     */
20     std::random_device rd;
21     std::mt19937 gen(rd()); // use a random device as the seed
22     std::uniform_int_distribution<> dist(1, max);
23     const int secret = dist(gen); // secret is in [1, max]
24     std::cout << "secret: " << secret << "\n"; // for debugging
25
26     while (tries > 0) {
27         /*
28         * (3) ask the player to enter a number
29         */
30         std::cout << "Enter a number\n";
31
32         /*
33         * (4) receive the guessed number
34         */
35         int guess = 0;
36         std::cin >> guess;
37
38         /*
39         * (5) compare the guessed number with the secret number
40         * and inform the player of the result
41         */
42         if (guess == secret) {
43             std::cout << "You Win!\n";
44             break; // terminate the loop early
45         } else if (guess > secret) {
46             std::cout << "guess > secret\n";
47         } else {
48             std::cout << "guess < secret\n";
49         }
50     }
51 }
```

```

51      * (6) loop: repeat until the player wins or loses
52      */
53      tries--;
54      // tries = tries - 1;
55  }
56
57  return 0;
58  }

```

其中，第 19-22 行使用[标准库 <random>](#) 生成随机数。随机数生成器是计算机科学中一个复杂且重要的话题。作为初学者，不要试图自己实现随机数生成器（如下图），而应直接使用标准库提供的功能。

```

int getRandomNumber()
{
    return 4; // chosen by fair dice roll.
              // guaranteed to be random.
}

```

? 该程序仍有不完善之处。你能找出来吗（不需要用 C++ 语言实现出来，仅从游戏逻辑角度思考）？

5.4 ChatGPT

ChatGPT 的推理与代码能力已经非常恐怖。猜数游戏对它来说早已是小菜一碟。我们不妨让它直接生成一段带界面的猜数游戏。代码主体如下：

代码块

```

1  void MainWindow::startGame() {
2      maxNumber_ = maxNumberSpin_>value();
3      resetRound();
4      std::uniform_int_distribution<int> dist(1, maxNumber_);
5      secret_ = dist(rng_);
6      active_ = true;
7      setMessage("A secret number is ready. Make your first guess!");
8      updateUi();
9      guessEdit_>setFocus();
10 }
11
12 void MainWindow::resetRound() {
13     attempts_ = 0;

```

```

14     guessEdit_>clear();
15 }
16
17 void MainWindow::submitGuess() {
18     if (!active_) return;
19     bool ok = false;
20     const int guess = guessEdit_>text().toInt(&ok);
21     if (!ok || guess < 1 || guess > maxNumber_) {
22         setMessage(QString("Please enter an integer from 1 to
%1.").arg(maxNumber_));
23         return;
24     }
25     ++attempts_;
26     if (guess == secret_) {
27         active_ = false;
28         setMessage(QString("🎉 Correct! The number was %1. You solved it in %2
attempt(s)!").arg(secret_).arg(attempts_), true);
29     } else if (guess < secret_) {
30         setMessage("Too low. Try a larger number.");
31     } else {
32         setMessage("Too high. Try a smaller number.");
33     }
34     guessEdit_>clear();
35     updateUi();
36 }
37
38 void MainWindow::giveUp() {
39     if (!active_) return;
40     active_ = false;
41     setMessage(QString("The hidden number was %1. Start a new game to try
again.").arg(secret_));
42     updateUi();
43 }

```

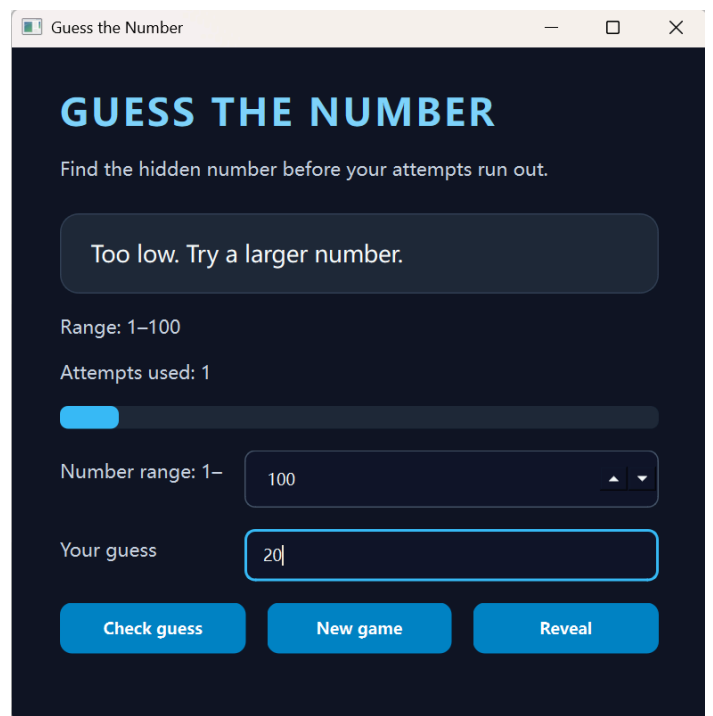


GuessNumberQt.exe

159.49KB



可执行程序



❗ 直接使用 AI 工具完成 OJ 作业属于抄袭行为！即使侥幸没被抓到，你也失去了一次训练的机会，“总归是要还的”。

（注：部分内容可能由 AI 生成）